Documentación técnica: Firebase y aplicación Android

La Migaja De San Carlos

Fecha de documentación: 30 de octubre de 2026.

Tecnologías: Java 8, Jersey, Tomcat 9, Oracle Cloud, Cloud Firestore, Firebase Cloud Messaging, Android Studio, Kotlin y Jetpack Compose.

Objetivo: describir la integración, las estructuras de datos, los endpoints, la seguridad y las pruebas realizadas para el seguimiento técnico y la presentación universitaria.

Seguridad: este documento no contiene contraseñas, llaves privadas ni tokens reales.

1. Resumen ejecutivo y alcance

La Migaja De San Carlos es un sistema de gestión de panaderia con una aplicación web administrativa y una aplicación Android dirigida al trabajo de vendedores en ruta. El backend Java es la puerta de entrada comun: autentica, autoriza y comunica las operaciones con Oracle Database y Firebase.

Responsabilidades por componente

Componente	Responsabilidad	Usuarios
Web local y HTTPS	Productos, recetas, inventario, pedidos, producción, despachos y gestión de rutas.	Administrador y roles operativos
Android Kotlin	Inicio de sesión, consulta de rutas asignadas, confirmacion de entregas y recepcion de avisos.	VENDEDOR_RUTA
Oracle Database	Datos transaccionales de negocio: pedidos, sucursales, usuarios, producción y despachos.	Acceso desde backend
Cloud Firestore	Estado operativo de rutas, vehiculos y dispositivos registrados para FCM.	Acceso administrativo desde backend
FCM	Aviso al vendedor cuando una ruta es asignada.	Dispositivo Android vinculado

Funciones verificadas durante el desarrollo

• Ingreso con JWT por HTTPS; los permisos de ADMINISTRADOR y VENDEDOR_RUTA se validaron con pruebas de acceso.

• Registro de rutas a partir de despachos Oracle, asignación de vendedor, asignación de vehículo y reporte de entrega.

• Alta y baja del registro FCM al iniciar/cerrar sesión desde un Samsung; mensaje push recibido al asignar una ruta con la app en segundo plano.

• Página **Gestión de rutas** publicada en HTTPS; asignación de vehículo desde interfaz web y reflejo posterior en Android.

Fuera del alcance de esta versión

No se implementaron flujos de incidentes de ruta ni reasignaciones. La app movil conserva internamente algunas vistas previas para otros roles, pero la demostración acordada se centra en vendedores. No se implemento la notificación visual personalizada con la app abierta en primer plano.

2. Arquitectura de integración

La aplicación movil y la web consumen la misma API REST de Java/Jersey. Ninguna de las dos debe incluir credenciales administrativas de Oracle ni una cuenta de servicio Firebase.

Arquitectura: Web administrativa y Android consumen la API Java en Tomcat sobre HTTPS. El backend consulta Oracle Database, administra las rutas y los dispositivos en Cloud Firestore y solicita avisos mediante Firebase Cloud Messaging.

Dirección HTTPS de producción

`https://lamigajasancarlos.duckdns.org/Proyecto_PanaderiaSanCarlos/webapi`

En desarrollo, el frontend web utiliza Tomcat local con la ruta `http://localhost:8080/Proyecto_PanaderiaSanCarlos/`. El entorno productivo esta publicado sobre HTTPS mediante Oracle Cloud y Caddy.

Flujo de una asignación

• El administrador autentica su sesión y utiliza la página web **Gestión de rutas**.

• El backend valida JWT, rol, despacho y vendedor mediante Oracle; después guarda la asignación en una transacción Firestore.

• Una vez confirmada la transacción, el backend consulta **dispositivos_fcm** y solicita a FCM el envio del aviso al vendedor.

• El vendedor abre Android y consulta sus rutas. Si tiene un vehículo asignado y la ruta esta lista, puede reportar la entrega.

Separacion entre Oracle y Firestore

Oracle conserva el ciclo de negocio del despacho (`DESPACHADO` y `RECIBIDO`); **Firestore** conserva el estado operativo de la ruta (`ENTREGA_REPORTADA`, etc.). Reportar una entrega desde Android no cambia automaticamente el despacho Oracle a RECIBIDO.

3. Modelo de datos en Cloud Firestore

Proyecto Firebase utilizado: **lamigajasancarlos**. Las colecciones documentan información vinculada con IDs existentes en Oracle. En los ejemplos siguientes se muestran valores de prueba, no credenciales.

Colección rutas

Campo	Ejemplo / significado
Documento	**despacho_6** (identificador de ruta)
idDespachoOracle / idPedidoOracle	**6 / 42** (referencias a Oracle)
sucursal / idVendedor	**CHIVA / 43**
idVehiculo	**vehiculo_demo_01**
estado / versión	**ENTREGA_REPORTADA / 4** (resultado final verificado)
actualizadoEn / entregaReportadaEn	Marcas temporales del servidor; entregaReportadaEn al reportar.

Ciclo de vida de una ruta

Secuencia de estados: PENDIENTE_ASIGNACION → PENDIENTE_VEHICULO → LISTA_PARA_ENTREGA → ENTREGA_REPORTADA.

Los cambios usan versión esperada para impedir que un cliente trabaje con un estado obsoleto. El backend rechaza la operación si no coincide la versión o el estado.

Colección vehiculos

Campo	Valor de ejemplo final
Documento	vehiculo_demo_01
nombre / placa / tipo	Vehículo de prueba / DEMO-001 / CAMIONETA
activo / estado	true / DISPONIBLE
versión	5 después de reportar despacho_6
idRuta	Ausente al liberarse el vehículo

Colección dispositivos_fcm

Campo	Uso
Documento	Huella SHA-256 del token (ID del documento).
idUsuario / plataforma	43 / ANDROID; asociacion con vendedor autenticado.
token / activo	Registro privado de FCM / true. No publicar.
actualizadoEn	Fecha de actualizacion del servidor.

La colección de dispositivos queda sin el documento del vendedor al desvincular correctamente el Samsung desde la aplicación.

4. Operaciones REST y validaciones

Rutas consumidas por la web y Android bajo `/webapi`. La identidad y el rol se extraen del JWT verificado; Android no decide por si mismo la identidad autorizada.

Método y recurso	Uso y permiso
POST /usuarios/login	Autenticar con cuenta Oracle y obtener JWT.
GET /rutas	Administrador ve rutas; vendedor recibe solo las rutas que le pertenecen.
POST /rutas	Registrar ruta de un despacho valido. Solo administrador.
PUT /rutas/{id}/asignar-vendedor	Asignar vendedor y actualizar estado/versión. Solo administrador.
PUT /rutas/{id}/asignar-vehículo	Asignar vehículo disponible, con versión de ruta y vehículo. Solo administrador.
PUT /rutas/{id}/reportar-entrega	Reportar entrega de ruta propia y liberar vehículo. Solo vendedor.
POST /dispositivos-fcm/registrar	Registrar token de Android del vendedor autenticado.
POST /dispositivos-fcm/desvincular	Desvincular ese token del vendedor autenticado.

Campos JSON utilizados

Asignar vehículo requiere las versiones de ambos documentos, para mantener la consistencia de la transacción:

```
{"idVehiculo":"vehiculo_demo_01",
"versionRutaEsperada":2,"versionVehiculoEsperada":3}
```

Reportar entrega envia solo la versión esperada; el ID del vendedor se extrae del JWT:

```
{"versionEsperada":3}
```

Registro FCM recibe el token del dispositivo, pero nunca acepta el idUsuario enviado por Android para decidir a quien pertenece.

Códigos HTTP relevantes

Codigo	Significado
200	Operación aceptada y resultado informado.
400	JSON o datos invalidos.
401	No autenticado o JWT ausente/invalido.
403	Rol sin permiso.
409	Conflicto de versión, estado o propiedad de la ruta.
503	Servicio o infraestructura no disponible.

5. Notificaciones Firebase Cloud Messaging

Registro de dispositivos

Android obtiene un token FCM usando Firebase Messaging después del inicio de sesión. **DispositivoFcmApi.kt** lo envia con el JWT a `POST /dispositivos-fcm/registrar`. **DispositivoFcmController.java** comprueba el rol y obtiene el usuario desde **SesionAutenticada**. **RegistroFcmManager.java** guarda el documento en Firestore con una huella del token como ID.

Desvinculación al cerrar sesión

Antes de cerrar sesión, Android invoca `POST /dispositivos-fcm/desvincular`. El backend verifica el propietario en una transacción y elimina el documento del dispositivo. Si la operación falla, la interfaz conserva la sesión para permitir reintentar, en vez de afirmar que se desvinculo correctamente.

Envio automatico de un aviso

RutaFirestoreManager.asignarVendedor(...) realiza primero la transacción de asignación. Una vez confirmada, invoca **NotificacionRutaFcmManager.notificarRutaAsignada(...)**. Este manager busca los dispositivos activos del vendedor 43 y utiliza Firebase Admin Messaging para enviar el aviso Nueva ruta asignada.

Momento	Comportamiento
Asignación valida	Firestore pasa a PENDIENTE_VEHICULO y aumenta la versión.
Después del commit	FCM intenta enviar a los tokens asociados al vendedor.
Error de FCM	La asignación no se revierte. Se registra una advertencia sin exponer el token.
La app esta en segundo plano	FCM/Android muestra normalmente la notificación; probado en el Samsung.
La app esta en primer plano	El servicio actual registra el evento; no implementa aviso visual personalizado.
Sesión cerrada	La desvinculación elimina el destinatario activo.

Limites operativos

La entrega de una notificación depende de Internet, permisos, ajustes de bateria y estado del sistema operativo. Forzar detencion puede impedir nuevas notificaciones hasta volver a abrir la app. Las renovaciones de token se detectan en **onNewToken**, pero el registro inmediato del token renovado no esta automatizado en esa funcion; se actualiza al iniciar sesión.

6. Aplicación Android con Kotlin y Jetpack Compose

La app se desarrollo en Android Studio con paquete `com.lamigaja.sancarlos` y UI declarativa Jetpack Compose + Material 3. La demostración principal utiliza el rol **VENDEDOR_RUTA**.

Archivos funcionales principales

Archivo	Responsabilidad
MainActivity.kt	Punto de entrada de la app; solicita permisos de notificaciones segun plataforma.
PantallaLogin.kt	Login HTTPS, JWT, perfil, navegacion, registro FCM y cierre de sesión seguro.
PantallaMisRutas.kt	GET de rutas, tarjetas y estados, dialogo de confirmacion, PUT de entrega y actualizacion.
DispositivoFcmApi.kt	Obtener token FCM y llamar registrar/desvincular por HTTPS.
LaMigajaMessagingService.kt	Recibir eventos de FCM y registrar mensajes en Logcat.
ui.theme/Color.kt y Theme.kt	Identidad visual crema, cafe y caramelo; sin colores dinamicos por defecto.

Recorrido del vendedor

• Escribe usuario y contraseña; la app solicita un JWT al backend HTTPS.

• Si el rol es VENDEDOR_RUTA, navega a Mis rutas y registra el token FCM. Un fallo temporal del registro no anula un login valido.

• Consulta solo sus rutas; las reportadas se muestran como historial y las pendientes de vehículo no permiten confirmar.

• Si la ruta esta en LISTA_PARA_ENTREGA, aparece el boton Confirmar entrega. Se muestra un dialogo antes de enviar el PUT.

• La app envia versionEsperada, refresca las rutas e informa los resultados; si hay error de conexion, se recomienda comprobar el estado antes de repetir.

Diseño visual

La interfaz utiliza tonos cafe, crema, caramelo y marfil, tarjetas por ruta, indicadores de estado, contadores de rutas y confirmaciones visuales. Las pantallas Productos y Despachos se conservan en el codigo para otros roles, pero no forman parte de la presentacion movil acordada.

7. Seguridad, control de acceso y consistencia

Identidad y roles

La autenticación se realiza en Java contra los usuarios de Oracle. Se entrega un JWT; **FiltroJwt** valida las solicitudes protegidas y **SesionAutenticada** representa al usuario autenticado. El vendedor no puede administrar rutas ni consultar todas las cuentas.

Rol	Puede hacer	No puede hacer
ADMINISTRADOR	Registrar y asignar rutas; consultar información administrativa.	Reportar una entrega haciendose pasar por vendedor.
VENDEDOR_RUTA	Consultar rutas propias, reportar entrega propia y registrar/desvincular FCM.	Asignar vendedores, asignar vehiculos, listar usuarios.

Concurrencia y validación

• Las transacciones de Firestore comprueban estado y versiones antes de escribir.

• Asignar vehículo cambia conjuntamente ruta y vehículo, para evitar asignaciones inconsistentes.

• Reportar entrega cambia conjuntamente la ruta a ENTREGA_REPORTADA y el vehículo a DISPONIBLE.

• Los despachos y sucursales se contrastan con Oracle, no con datos no confiables del cliente.

Proteccion de secretos

• La cuenta de servicio Firebase Admin y la configuración .env estan fuera del repositorio y del WAR, bajo una carpeta privada en el servidor.

• Android contiene configuración de cliente Firebase, pero **nunca** debe recibir el JSON de cuenta de servicio ni contrasenas administrativas.

• No subir a GitHub: .env real, firebase-admin.json, archivos .key/.pem, keystores, JWT, tokens FCM, bases de datos privadas ni WAR de producción.

• Revisar con git status y git diff --cached --name-only antes de cada commit; ignorar target/ y nbproject/private/.

Reglas Firestore: el cliente no escribe directamente las colecciones administrativas; las operaciones pasan por la API Java con Firebase Admin y sus validaciones. Esto no reemplaza una revision periodica de reglas y permisos en producción.

8. Pruebas efectuadas y evidencias

Los resultados siguientes reflejan las comprobaciones realizadas y comunicadas durante la construccion del proyecto, incluyendo capturas de Firebase, PowerShell, web HTTPS y Android en un Samsung. No equivalen a un proceso automatizado de QA ni a una auditoria externa.

Prueba	Resultado observado
JWT en rutas	401 sin token; 403 al intentar reportar con administrador; 400 por datos incompletos; 409 por versión incorrecta.
Registro FCM	401 / 403 / 400 / 400 en Cloud, segun autenticación, rol y datos.
Desvinculación FCM	401 / 403 / 400 / 400 en Cloud; el documento de dispositivo desaparecio tras cerrar sesión.
Despacho 3	Ruta ENTREGA_REPORTADA versión 4; vehículo DISPONIBLE versión 3.
Despacho 6	Pedido Oracle 42, sucursal CHIVA; asignación de vendedor 43 con push recibido.
Gestión web de rutas	Asignación de vehículo desde HTTPS; la ruta quedo LISTA_PARA_ENTREGA, versión 3.
Reporte Android de despacho 6	Ruta ENTREGA_REPORTADA versión 4; vehículo DISPONIBLE versión 5.
Android visual	Pantalla Mis rutas con historial de despachos 3 y 6; tarjetas, contadores y tema corporativo.

Estados finales de las pruebas

Recurso	Estado al cierre de las pruebas
rutas/despacho_3	ENTREGA_REPORTADA / versión 4
rutas/despacho_6	ENTREGA_REPORTADA / versión 4
vehiculos/vehiculo_demo_01	DISPONIBLE / versión 5; sin idRuta
dispositivos_fcm	Registro activo del Samsung tras volver a iniciar sesión; depende de la sesión actual.

Comprobación adicional recomendada para defensa

Crear un producto y su receta desde cero, probar falta de insumos, abastecimiento autorizado, pedido, producción, despacho, asignación web, recepcion FCM y reporte en Android. Guardar evidencias del antes/después de inventario y de la respuesta a intentos no autorizados.

9. Operación, despliegue y mantenimiento

Entorno web y backend

Elemento	Ubicacion / consideracion
Servidor	Oracle Cloud Infrastructure, Oracle Linux y Tomcat 9, con HTTPS por Caddy.
WAR publicado	/opt/tomcat/webapps/Proyecto_PanaderiaSanCarlos.war
Copias de seguridad	/opt/tomcat/respaldo/ (guardar WAR y directorio expandido antes de actualizar).
Configuración privada	/opt/tomcat/.panaderia-san-carlos/ (no compartir contenido).
Compilación	NetBeans: Clean and Build; proyecto Maven, Java 8, salida target/*.war.
Validación de despliegue	SHA-256, unzip -t, systemctl is-active tomcat y HTTP 200 local/HTTPS.

Android Studio

Abrir el proyecto Android de La Migaja, sincronizar Gradle, comprobar la configuración cliente de Firebase, compilar con Build > Make Project e instalar mediante Run en un teléfono de prueba. Se utilizo un Samsung real. Para recibir avisos, verificar Internet, permisos de notificaciones y cuenta de vendedor activa.

Control de versiones recomendado

El repositorio GitHub privado se utiliza para mantener el codigo fuente web y Java. Guardar esta documentación bajo `docs/Documentacion_Firebase_Android_La_Migaja.pdf`. Para incluir la app Android como codigo versionado, puede utilizarse un directorio separado `android/` dentro de un repositorio estructurado, o un repositorio Android independiente; no se debe copiar el proyecto Android a ciegas dentro de src/main/webapp.

Checklist previo a cada push

• Revisar rama y remoto: git branch --show-current; git remote -v.

• Comprobar .gitignore y el conjunto de archivos preparados: git status --short; git diff --cached --name-only.

• Excluir target/, .env y firebase-admin.json; comprobar también archivos ya rastreados.

• Crear commit descriptivo y usar git push origin main una vez verificada la seleccion.

Precauciones con notificaciones

El servicio FCM confirma la aceptacion del envio, no garantiza que el teléfono lo muestre al instante. El manejo inmediato de cambios de token en onNewToken es una posible mejora futura; no es necesario para el alcance de la demostración ya aprobada.

10. Guion para la presentacion y preguntas frecuentes

Demostración integral sugerida

Momento	Explicacion sugerida
1. Catalogo, receta, inventario	Un pan terminado se elabora a partir de insumos definidos por receta.
2. Pedido sucursal	Una sucursal solicita panes por turno y envia su pedido.
3. Producción	Se validan insumos; iniciar consume inventario y finalizar registra el producto fabricado.
4. Despacho	El pedido listo se despacha desde Oracle; se obtiene un ID de despacho.
5. Administración de rutas	Desde la web HTTPS se registra ruta, asigna vendedor y vehículo.
6. Firebase + Android	FCM notifica al vendedor; Android consulta sus rutas y confirma la entrega.
7. Verificación	Firestore refleja ENTREGA_REPORTADA y libera el vehículo; Oracle mantiene su ciclo propio de recepcion.

Cinco preguntas que pueden hacerte

¿Por qué Oracle y Firebase? Oracle centraliza las transacciones de la panaderia; Firestore modela el trabajo de rutas y dispositivos para avisos moviles.

¿Android se conecta directamente a Oracle? No. Todo pasa por HTTPS y la API Java, que valida JWT y permisos.

¿Cómo evitan asignar el mismo vehículo dos veces? Se consulta estado, versión y relacion con ruta en una transacción Firestore.

¿Qué ocurre si falla FCM? La asignación queda guardada; el envio del aviso es posterior al commit y no revierte el trabajo confirmado.

¿Reportar entrega significa que sucursal recibio el despacho? No. El reporte del vendedor en Firebase y la recepcion Oracle son etapas diferentes.

Referencias oficiales de consulta

Firebase Admin SDK - Configuración

FCM - Recepcion de mensajes en Android

FCM - Gestión de registros y tokens

Android - Jetpack Compose

Documento preparado para: repositorio del proyecto, seguimiento técnico y práctica de defensa universitaria. Actualizar tras cambios importantes en el codigo o las reglas de Firebase.